Full Educational Platform Investigation

Repository: `Manasa`
Investigation date: 2026-10-23
Method: Static review of first-party source, views, models, migrations, project/configuration files, and non-vendored frontend assets. No application, database, tests, profiler, or deployment was run.

1. Executive Summary

This is an early ASP.NET Core MVC application on .NET 9, using ASP.NET Core Identity and EF Core with SQL Server. The implemented scope is account registration/authentication and administrator management of students, teachers, subjects, courses, and course packages, with a record-count dashboard.

It does not yet implement learning-content delivery, lessons, enrollment, purchase/payment, access grants, assessments, grading, progress, notifications, learner dashboards, or teacher operational workflows. Package price and access duration are catalog fields only; no code consumes them to grant access.

Static review found **2 confirmed user-facing/functional gaps** (F-01, F-02) and **7 material risks or policy questions** (R-01–R-07). No P0 issue was confirmed. Findings below distinguish defects from risks; priorities are source-based, not runtime-severity measurements.

2. Project Overview and Technology Stack

Area	Evidence	Purpose
Runtime	`Manasa.csproj`	ASP.NET Core MVC, `net9.0`, nullable enabled
Identity	`Manasa.csproj`, `Program.cs`	Identity EF stores, roles, cookie authentication
ORM/database	`Manasa.csproj`, `Program.cs`	EF Core SQL Server 9.0.21
UI	`Views/`, `Areas/Admin/Views/`, `wwwroot/`	Razor MVC, Bootstrap, jQuery, unobtrusive validation
Persistence	`Models/`, `Data/ApplicationDbContext.cs`	Identity plus catalog entities and relationship configuration
Migrations	`Data/Migrations/`	Five migration steps, designer metadata and model snapshot
External services	Repository inventory	No payment, mail, file-storage, analytics, or LMS client found
Tests	Repository inventory	No test project or test files found

The only checked-in connection string is in `appsettings.Development.json` and targets local SQL Server LocalDB. Production connection configuration must come from deployment configuration; that contract is not documented here. `appsettings.json` sets logging and `AllowedHosts: *`.

3. Architecture and Project Structure

Path	Responsibility
`Program.cs`	Service registration, Identity cookie configuration, development-only migrations/roles/seeding, middleware and conventional MVC routes.
`Controllers/`	Public Home/Privacy/Error and account registration, login, logout, password setup.
`Areas/Admin/Controllers/`	Administrator-only dashboard and student/teacher/subject/course/package management.
`Areas/Admin/Views/`	Admin screens, forms, navigation and TempData feedback.
`Views/`	Public/account pages, shared layout and error page.
`ViewModels/`	Form/display contracts and DataAnnotations validation.
`Models/`	Identity user and Subject, TeacherProfile, SubjectTeacher, Course, CoursePackage.
`Data/ApplicationDbContext.cs`	Identity DbContext, entity relationships, indexes, delete behavior and seed catalog data.
`Data/DevelopmentDataSeeder.cs`	Development account seeding when configured passwords exist; demo teacher/profile linkage.
`Data/Migrations/`	Initial schema, username sequence, roles, unique email constraint, user CreatedAt timestamp.
`wwwroot/css/site.css`	Starter and admin styling with responsive breakpoints.
`wwwroot/js/site.js`	Empty starter script.
`wwwroot/lib/`	Vendored Bootstrap, jQuery and validation libraries.
`Properties/launchSettings.json`	Local launch profiles.

Request flow: MVC routes map requests to controllers; the Admin area defaults to `Admin/Dashboard/Index`. Authentication precedes authorization. Admin controllers apply `[Authorize(Roles = "Administrator")]`; mutation forms use antiforgery validation. Controllers validate view models, access Identity/EF directly, save changes, and render or redirect. There is no separate service or API layer; business rules reside in controllers, Identity, and EF configuration.

In `Program.cs`, migrations, role creation, and demo seeding run only in Development. Production migration and initial role/account provisioning are therefore deployment responsibilities. Non-development uses `/Home/Error` exception handling and HSTS; HTTPS redirection is always enabled.

4. Complete Feature Inventory

Feature	Implementation	Status / limitation
Public home/privacy/error	`HomeController`, `Views/Home/*`, `Views/Shared/Error.cshtml`	Starter template; home is not a course catalog.
Student registration	`AccountController.Register`, `RegisterViewModel`, Identity user/Student role, SQL sequence username	Implemented; no email verification; terms acceptance is discarded (F-02).
Login/logout	`AccountController.Login/Logout`, Identity cookie	Username/email login, generic failure message, lockout-on-failure, local return URL check, antiforgery logout. No MFA/recovery.
Initial password setup	`AccountController.SetPassword`; admin creates passwordless accounts and reset tokens	Implemented with manual link sharing; no mail delivery or resend workflow. Completion copy is stale (F-01).
Roles/authorization	Role seed and Admin controller attributes	Administrator area protected. Teacher/Student have no role-specific feature area.
Student administration	`StudentsController` and views	Search, 25-item paging, create/edit, active toggle; no delete/bulk/audit/self-service learning.
Teacher administration	`TeachersController`, `TeacherProfile`, `SubjectTeacher`	Create/edit profile/account and subject links; toggle active; no teacher workspace.
Subject administration	`SubjectsController`, `Subject`	Search, create/edit, ordering/active state, guarded delete.
Course administration	`CoursesController`, `Course`	Search/filter, create/edit, publish flag, subject/teacher validation, guarded delete. No lessons or learner-facing access.
Package administration	`CoursePackagesController`, `CoursePackage`	Search/filter, create/edit/delete; no transaction, enrollment, entitlement, expiry enforcement or payment.
Admin dashboard	`DashboardController` and view models	Counts plus six recent courses.

Feature	Implementation	Status / limitation
Client validation	Shared validation partial and bundled libraries	Included in forms; browser behavior not run. Server validation also exists for annotated fields.
Seed data	`ApplicationDbContext.OnModelCreating`, `DevelopmentDataSeeder`	Two subjects, sample unpublished course, zero-priced sample package; optional dev accounts gated by configured password keys.

Countable summary: **13 feature groups** in the table and **6 major workflows**: registration, login/logout, initial password setup, student management, teacher management, and subject/course/package catalog administration. Dashboard is an additional read workflow.

5. End-to-End Workflows and Business Cycles

Registration

Anonymous GET renders the form. Antiforgery-protected POST validates DataAnnotations, checks email, begins a database transaction, obtains `NEXT VALUE FOR dbo.StudentUsernameSequence`, creates the Identity account and Student role, and commits. Identity errors are added to ModelState; role assignment failure rolls back and logs error codes; outer exceptions return a generic error. Unique normalized-email and username indexes backstop duplicates. There is no email confirmation and no persisted consent evidence.

Login and logout

Login POST resolves username then email and calls `PasswordSignInAsync` with lockout-on-failure. Unknown user and failed password share a generic response. Only local return URLs are honored. Auth cookie expires after eight hours with sliding renewal; Logout is authorized POST with antiforgery. Explicit password complexity, lockout threshold/duration, confirmation, and MFA policy are not configured in `Program.cs`.

Initial password setup

Admin creates a passwordless student/teacher, commits account/profile/role, generates an Identity reset token, and exposes a setup URL in TempData for manual sharing. Anonymous GET checks the user exists and has no password. POST checks again and calls `ResetPasswordAsync`, which validates the token. Invalid user/already-set account returns 404; invalid token produces Identity errors. Token generation happens after the account transaction commits, so a failure then can leave an account without a delivered link (R-04). No email or resend process exists.

Student management

Admin-only list filters users by Student role, searches, counts, sorts and pages 25 rows. Creation atomically adds user and role, then creates setup token/link after commit. Edit constrains the target to Student role and updates display name/email. Toggle sets/clears a lockout end about 100 years ahead. The list infers active state from `LockoutEnd`, conflating administrative deactivation and temporary failed-login lockout (R-07). No deletion is implemented.

Teacher management

Create validates unique email and active subject IDs; transaction creates Identity user, Teacher role, profile and subject links. Edit changes profile and account fields and reconciles subject links. Toggle changes both profile `IsActive` and Identity lockout. Setup link is generated after commit. Exceptions in most admin save/token paths are not translated to a friendly recovery path. There is no teacher-facing authoring or student-management workflow.

Catalog management

Subjects can be searched, created, edited and deleted only when not referenced. Courses can be searched by title and filtered by subject/publication; validation requires an active subject and an eligible teacher assignment, then saves. Delete checks for packages and assigned teacher. Packages validate course and view-model ranges, then create/edit/delete. Foreign keys use restrictive deletion. Concurrent reference changes can still make precheck/delete fail at the database, and most such exceptions are uncaught. Package data does not create an enrollment or access grant.

6. Bugs and Confirmed Defects

ID	Priority	Description / evidence	Impact and recommendation	Confidence
F-01	P3	`Views/Account/SetPasswordComplete.cshtml:5` says login will be available “when the login page is available”, but `AccountController.Login` exists at `Controllers/AccountController.cs:80-117` and shared navigation links to it.	Misleading post-setup guidance. Update copy to point to existing sign-in.	Confirmed
F-02	P2	`ViewModels/Account/RegisterViewModel.cs:28-30` requires `AcceptTerms`; `Controllers/AccountController.cs:41-50` creates an `ApplicationUser` without storing acceptance, timestamp, or version; `Models/ApplicationUser.cs` has no consent fields.	No evidence can be retrieved about which terms were accepted. Clarify legal/product requirement; if required, persist version and timestamp transactionally.	Confirmed implementation gap; impact depends on policy

No confirmed P0 or P1 defect was established by source review. An initial concern about production exception handling was checked against `Program.cs`: exception middleware and HSTS are present for non-development environments, so it is not reported as a production defect.

7. Potential Risks and Areas Requiring Verification

ID	Priority	Evidence / concern	Needed verification	Confidence
R-01	P1	`DevelopmentDataSeeder.cs:16-30` provisions two development administrator identities when corresponding config password values exist; seeding only runs in Development (`Program.cs`).	Confirm deployed environment classification and that demo accounts/secrets cannot reach public deployments. No password value is reproduced.	High confidence risk
R-02	P1	`SetPassword` is anonymous and reset token is carried in a manually shared URL (`AccountController.cs:126-163` , admin create actions).	Verify Identity token lifetime, HTTPS, access-log/referrer/analytics handling, and recipient controls. Treat URL as bearer credential.	Needs verification
R-03	P1	Public registration provisions the Student role; no rate limiter is configured in `Program.cs` .	Confirm open-registration policy and abuse controls/rate limiting.	High confidence observation
R-04	P2	Student/teacher account commits precede setup token generation (`StudentsController.cs:80-112` , `TeachersController.cs:52-95`).	Add and verify a recoverable resend/reset workflow; failure after commit can strand passwordless account.	High confidence risk
R-05	P2	Identity security policy is mostly implicit; registration model min length is 6 (`RegisterViewModel.cs:19-21`), setup model min length is 8 (`SetPasswordViewModel.cs:13-14`).	Confirm effective framework defaults and define/align production password, lockout, confirmation and MFA policy.	Confirmed config observation; policy needs verification
R-06	P2	Student `IsActive` is calculated from `LockoutEnd` (`StudentsController.cs:43-51`); toggle also writes `LockoutEnd` (`169-183`).	Determine whether deactivation should be distinct from failed-login lockout and whether toggling can clear a security lock.	High confidence
R-07	P2	Teacher, subject, course and package list queries have no pagination (`TeachersController` , `SubjectsController` , `CoursesController` , `CoursePackagesController` Index methods).	Estimate catalog growth; add bounded lists if expected volume warrants it.	High confidence

8. Performance Findings

No profiling or benchmarking was performed.

ID	Priority	Evidence	Impact / recommendation
PERF-01	P2	Teacher, subject, course and package indexes materialize every matching row. Course index includes package entities to display their count.	Memory, query time and HTML can grow with catalog size. Paginate and project only needed columns; replace included package rows with aggregate count if query evidence supports it.
PERF-02	P2	Dashboard executes separate counts for student/teacher/subject/course/published-course/package plus a recent-course query (`DashboardController.Index`).	Multiple DB round trips per request; likely fine at small scale, but measure at expected load.
PERF-03	P2	Search uses `Contains` in student, teacher, subject, course, package paths.	Substring search may not benefit from ordinary B-tree indexes, particularly at scale. Measure query plans; paginate and consider search strategy if needed.
PERF-04	P3	Existing FK and identity indexes cover main joins; no explicit indexes for course title or common display/order/date filters are configured in `ApplicationDbContext`.	Do not add speculative indexes; inspect representative SQL plans and data volume first.

9. Security Findings

ID	Priority	Evidence	Recommendation
SEC-01	P1	Development seeder creates privileged demo/admin users when configured; `Program.cs` gates this to Development.	Verify environment separation and secret injection; never expose Development environment publicly.
SEC-02	P1	Anonymous token-based SetPassword flow; setup URL shared manually.	Review token lifetime and safe delivery/logging; provide a revocation/regeneration path.
SEC-03	P1	Public registration/login, with no explicit rate limiting in source. Login error is generic, which limits direct account enumeration.	Define signup policy and appropriate abuse monitoring/throttling.
SEC-04	P2	No email confirmation/recovery flow found.	Confirm whether verified email and self-service recovery are launch requirements.
SEC-05	P2	Password model constraints differ and Identity policy is not explicitly configured.	Define one policy and align UI/server validation.
SEC-06	P3	Cookie is HttpOnly, SameSite=Lax, SecurePolicy Always outside Development, eight-hour sliding expiration (`Program.cs:15-25`).	Review data-protection key persistence, reverse-proxy HTTPS configuration, and session revocation in deployment.

No direct interpolated SQL risk was found: the sequence SQL is constant text. Admin POST actions are antiforgery protected. Razor output is generally encoded. No upload/download handlers or external credentials are present. No confirmed XSS, CSRF, or IDOR defect was established from inspected code.

10. Database and Data Integrity Findings

- Entities: Identity user, Subject, TeacherProfile, SubjectTeacher, Course, CoursePackage. One user has at most one teacher profile; teacher/subject is many-to-many; subject has courses; teacher has optional courses; course has packages.
- Foreign keys for catalog relationships use `Restrict`; Identity-owned relations generally cascade. Subject name, normalized email, normalized username, teacher UserId, and subject-teacher pair are unique. FK indexes exist.
- `CoursePackage.Price` uses decimal(18,2). Admin view-model ranges constrain package price/duration and teacher experience, but corresponding database check constraints are absent; non-MVC writers could bypass those rules.
- Deletion guards are prechecks plus restrictive FKs; concurrent reference changes can still produce database exceptions. Ordinary edits do not use concurrency tokens.
- Migrations: initial schema `20261021133521_InitialLearningPlatform`; sequence; role seed; unique email index; user CreatedAtUtc. Snapshot exists. Migrations were not executed.
- Seed catalog rows include two subjects, an unpublished sample course, and a zero-cost 30-day package with fixed 2026-01-01 dates. Development accounts are configuration-gated.
- No enrollment, order, payment, entitlement, lesson, assessment, progress, or audit schema exists. Thus the catalog package fields have no data-integrity link to user access.
- Course publication does not enforce assigned teacher, active package, or content prerequisites. Whether that is an inconsistency depends on product policy.

11. Frontend and User Experience Findings

Public Home is starter content and does not expose a public catalog. Admin layout shows TempData success/error messages; forms use labels, validation summaries/field messages, and shared client-validation scripts. Empty states exist in several admin lists. Admin CSS includes responsive layout rules and table wrappers, but no browser, keyboard, screen-reader, or device testing was performed.

F-01 is stale completion guidance; F-02 is non-persisted terms acceptance. Expected validation failures are usually displayed, while DB errors in most admin paths are not consistently converted to friendly errors. Teacher/Student users have no functional learning-facing screens. A few symbols appeared mojibake in console output; this may be terminal encoding and must be checked in a browser before calling it a defect.

12. API and External Integration Inventory

No API controllers, minimal APIs, Swagger/OpenAPI configuration, API DTO contracts, versioning, or external service client was found. The app serves MVC routes only.

Route family	Access	Purpose
`/`, `/Home/Index`, `/Home/Privacy`	Anonymous	Starter site
`/Account/Register`, `/Account/Login`	Anonymous	Student signup and login
`/Account/Logout`	Authenticated POST	Sign out
`/Account/SetPassword`	Anonymous; POST requires valid Identity token	Initial account password setup
`/Admin/Dashboard`, `/Admin/Students`, `/Admin/Teachers`, `/Admin/Subjects`, `/Admin/Courses`, `/Admin/CoursePackages`	Administrator	Dashboard and management screens/actions
`/Home/Error`	MVC route; production exception target	Error view

13. Missing or Incomplete Features

Not present in this checkout: lessons/material delivery, enrollment, purchase/payment, entitlement/access expiry, quizzes/exams, assignments/grading, progress, notifications, learner/teacher dashboards, teacher content authoring, reports, email delivery, email confirmation, password recovery/resend, audit history, public catalog, and automated tests.

Partially implemented: package price/duration and course publication are editable fields without a consumer-facing sale/access workflow; Teacher and Student roles exist without role-specific workspaces; initial setup links are manual rather than delivered. Do not infer these are product requirements; their priority requires product confirmation.

14. Testing and Code Quality Assessment

No tests, test project, CI configuration, or test scripts were found. No build or test command was run. Controller actions depend directly on EF and Identity; key cross-entity rules are implemented in controller helpers, while some input rules are DataAnnotations and some constraints are in SQL. Error handling varies: registration catches broad exceptions, while most admin persistence paths do not. Development startup migrates/seeds automatically; production migration and role provisioning instructions are absent. Vendored browser-library internals and dependency vulnerabilities were not audited.

Recommended focused tests: registration and role assignment/rollback; token setup success/expiry; teacher-subject-course validation; lockout/deactivation interaction; guarded deletions; unique email races; and admin authorization. No large refactor is recommended solely for style.

15. Prioritized Findings and Recommended Fixes

ID	Priority	Finding	Proposed action	Dependencies
R-01	P1	Dev seeder provisions privileged users when configured.	Verify environment isolation and secret handling.	Deployment owners
R-02	P1	Setup URL is a manually shared bearer token.	Verify lifetime/handling; add recoverable reissue path.	Account lifecycle policy
R-03	P1	Public signup has no explicit throttling.	Confirm signup policy; add appropriate abuse controls if open.	Product/security policy
F-02	P2	Accepted terms are not stored.	Clarify legal requirement; persist timestamp/version if required.	Legal/product decision, schema change
R-04	P2	Account commits before setup token/link generation.	Add admin recovery/resend behavior and operational logging.	Account lifecycle policy
R-05	P2	Password/lockout policy implicit and constraints differ.	Define and configure consistent policy.	Security policy
R-06	P2	Lockout and deactivation share state.	Separate or explicitly define state semantics.	Account state requirements
R-07	P2	Catalog lists are unpaged.	Add pagination based on expected volume and UX.	Scale/UX requirements
F-01	P3	Setup completion text claims login is unavailable.	Correct copy and link to sign-in.	None

Top-level priority counts: P0: 0; P1: 3; P2: 5; P3: 1. Counts represent the prioritized table above, including risks, not confirmed defects alone.

16. Recommended Development Roadmap

Phase 1 — Account and deployment safety

1. Verify development environment and secret isolation.
2. Confirm registration, email verification, throttling and terms-consent policies.
3. Define explicit password/lockout rules and distinguish deactivation from transient lockout.
4. Make setup-link delivery/reissue recoverable and review token handling.

Phase 2 — Workflow reliability

1. Provide consistent user-facing handling and logging for persistence failures.
2. Document production migration, role provisioning, backup and rollback ownership.
3. Add focused workflow and authorization tests.

Phase 3 — Learning workflows (after scope approval)

1. Define lesson/content model and publication rules.
2. Define purchase/enrollment/entitlement model before using package price/duration for access.
3. Implement learner discovery/access and teacher authoring against approved requirements.
4. Add assessments, progress, notifications and reporting only as requirements are confirmed.

Phase 4 — Scale and user experience

1. Paginate admin catalogs and review query plans with representative data.
2. Perform responsive/accessibility/manual browser review.
3. Add monitoring and audit needs based on operational policy.

17. Questions Requiring Product or Business Clarification

1. Is public student self-registration intended, or should accounts be invited?
2. Must terms acceptance be retained with version and timestamp?
3. Is email confirmation and self-service password recovery required?
4. Are teacher tools/content authoring in the current phase?
5. What event starts package access duration, and is payment intended?
6. Should publication require teacher, active package and learning content?
7. Should administrative deactivation be distinct from login-failure lockout, and what happens to active sessions?
8. Who owns production database migrations and initial role provisioning?
9. Are Home/Privacy starter pages intentional for Phase 1?

18. Investigation Scope, Limitations, and Unverified Areas

Reviewed all first-party source, Razor views, view models, project/configuration files, migration source/snapshots and non-vendored frontend assets in the repository inventory. Third-party library internals were not audited. No source/configuration was changed during investigation.

Not performed: build/run, database connection or migration execution, workflow execution, browser/device/accessibility review, penetration testing, dependency scan, load test, query-plan inspection, production configuration review, or external-service verification. No test suite exists in the inventory. No secret/password values are included. Runtime defaults, deployment configuration, token exposure and intended product behavior require the checks listed above.

File paths and line references are repository-relative with one-based line numbers. Conclusions are limited to this checkout; deployed configuration and database contents may differ.